

ENHANCING ROBOT NAVIGATION WITH VISION LANGUAGE MODELS

Advisor: Samet Güler

NISANUR TÜRKMEN- 2211011052
nisanur.turkmen@agu.edu.tr

AYŞE ECE AKKURT- 2211011006
ayseece.akkurt@agu.edu.tr

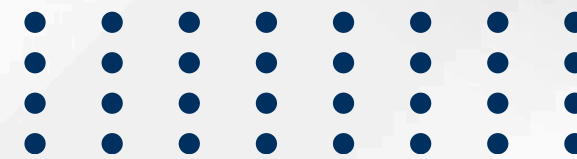
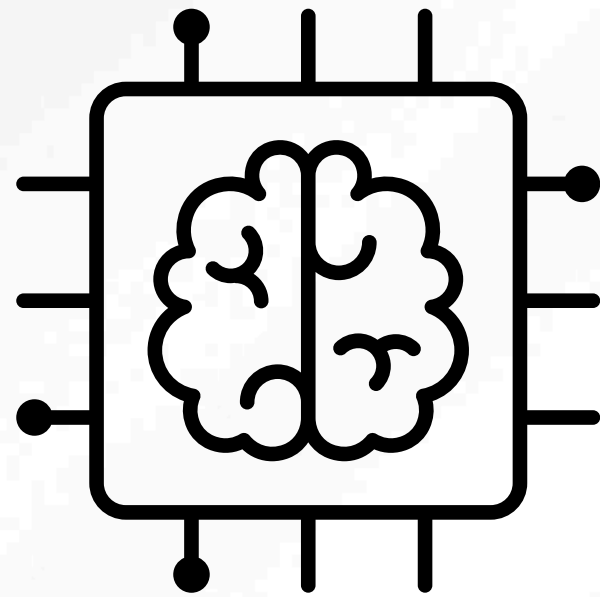


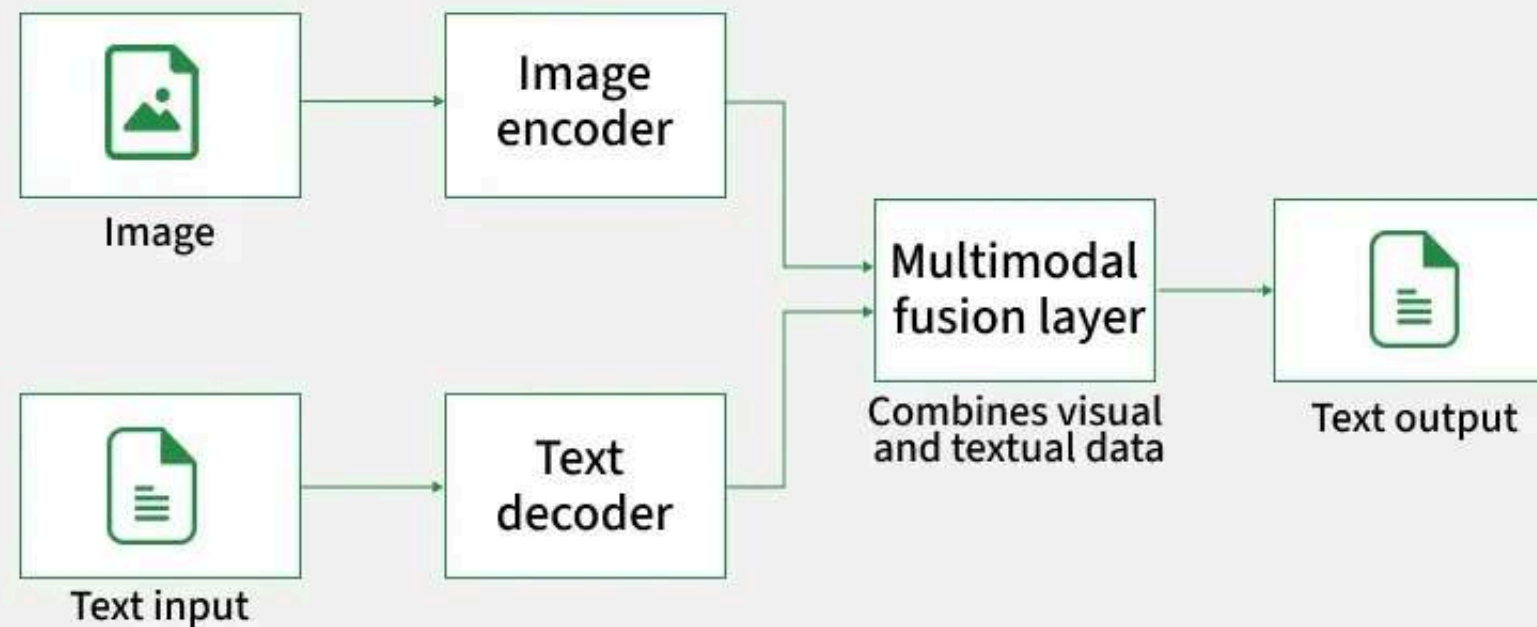
TABLE OF CONTENT



- ★ Project Description
- ★ Previous Implementations
- ★ Current System: Habitat-Sim + VLMaps
- ★ VLMaps Architecture
- ★ Language-to-Navigation Pipeline
- ★ Implementation Results
- ★ Failure Analysis
- ★ Performance Evaluation
- ★ Next Steps
- ★ Conclusion

PROJECT DESCRIPTION

The Structure of a VLM



Vision Language Models (VLMs) Architecture, GeeksforGeeks, 2024. Available: <https://www.geeksforgeeks.org/vision-language-models-vlms-explained/>

LIDAR & YOLO

- Limitations: They are not enough for navigation
- Traditional robots default understanding of human intent
- No interpretations and closed vocabulary

VLM

- Integrate VLM into perception-decision pipeline
- Rich vocabulary and reports the scene's 'story' in natural language.
- Understand context or human intention
- Open vocabulary and prompt based
- Semantic Oriented

SLAM: CONVENTIONAL VS. SEMANTIC UNDERSTANDING

CONVENTIONAL SLAM

ORB-SLAM · LSD-SLAM · RTAB-Map · Cartographer

LIMITATIONS

- **Geometry only**
Builds point clouds, occupancy grids, or feature maps. Knows where things are, not what they are.
- **No semantic awareness**
A chair and a trash bin look the same — just obstacles.
- **Can't follow language**
“Go to the kitchen” is meaningless to a SLAM map.
- **Closed-world assumption**
Works on what it was tuned for; new object classes need re-engineering.
- **No reasoning**
The map is a passive geometric description, not a queryable knowledge structure.

SEMANTIC UNDERSTANDING

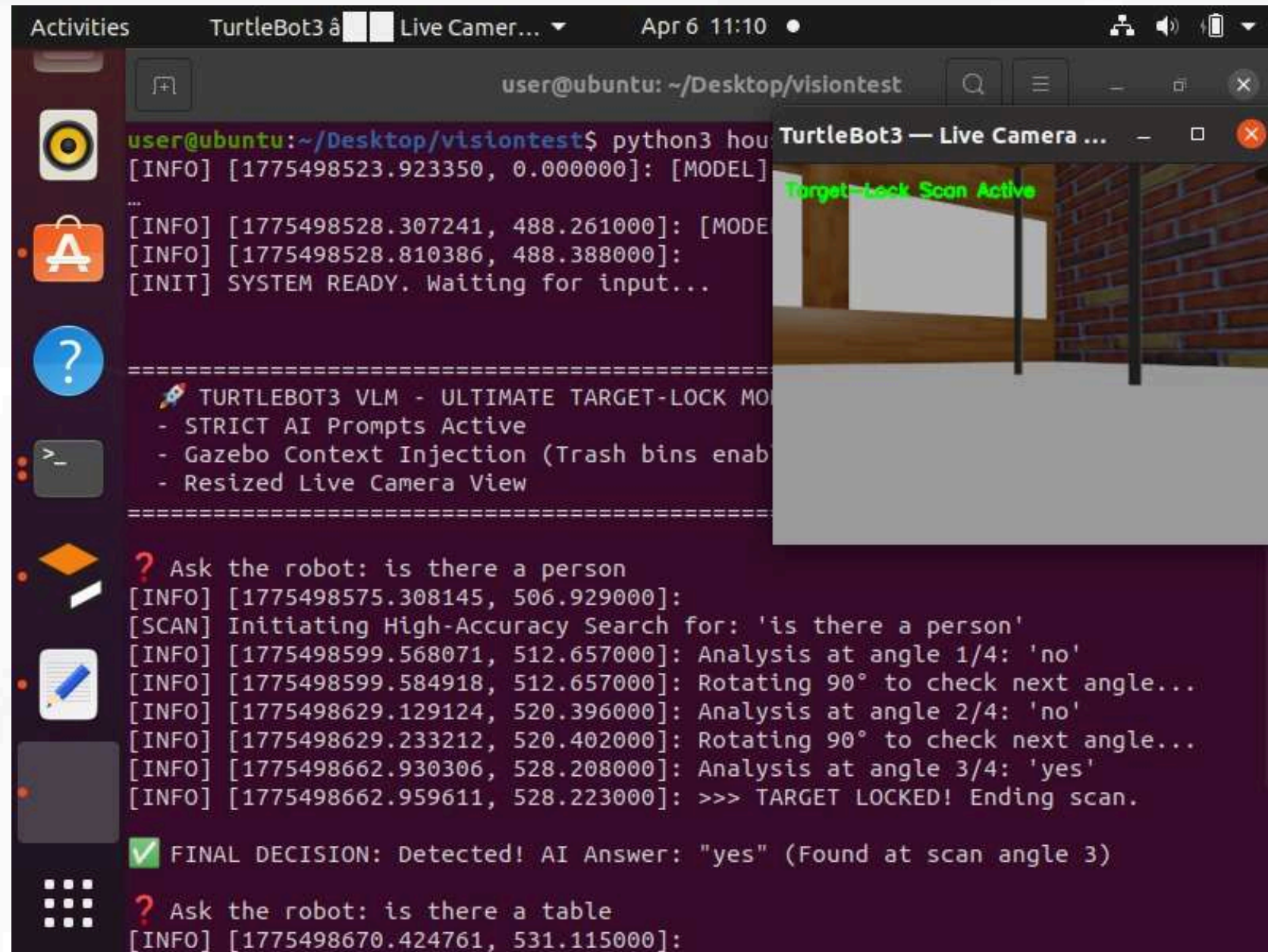
VLMaps · NLMap · OpenScene · ConceptFusion · LSeg

NEW DIRECTION

- **Pixels carry meaning**
Every map cell stores not just geometry, but a vision-language feature vector.
- **Open vocabulary**
Query the map with any word: “sofa”, “study desk”, “seating area”.
- **Language-driven nav**
Natural language commands map directly to spatial goals.
- **Cross-embodiment**
One semantic map serves drones, ground robots, and manipulators.
- **Zero-shot reasoning**
The robot solves new tasks without retraining.

PREVIOUS IMPLEMENTATION

DROPPED



The screenshot shows a Gazebo simulation environment. On the left, a terminal window displays the following logs:

```
user@ubuntu: ~/Desktop/visiontest
user@ubuntu:~/Desktop/visiontest$ python3 hou
[INFO] [1775498523.923350, 0.000000]: [MODEL]
...
[INFO] [1775498528.307241, 488.261000]: [MODE
[INFO] [1775498528.810386, 488.388000]:
[INIT] SYSTEM READY. Waiting for input...

=====
? TURTLEBOT3 VLM - ULTIMATE TARGET-LOCK MO
- STRICT AI Prompts Active
- Gazebo Context Injection (Trash bins enab
- Resized Live Camera View
=====

? Ask the robot: is there a person
[INFO] [1775498575.308145, 506.929000]:
[SCAN] Initiating High-Accuracy Search for: 'is there a person'
[INFO] [1775498599.568071, 512.657000]: Analysis at angle 1/4: 'no'
[INFO] [1775498599.584918, 512.657000]: Rotating 90° to check next angle...
[INFO] [1775498629.129124, 520.396000]: Analysis at angle 2/4: 'no'
[INFO] [1775498629.233212, 520.402000]: Rotating 90° to check next angle...
[INFO] [1775498662.930306, 528.208000]: Analysis at angle 3/4: 'yes'
[INFO] [1775498662.959611, 528.223000]: >>> TARGET LOCKED! Ending scan.

✓ FINAL DECISION: Detected! AI Answer: "yes" (Found at scan angle 3)

? Ask the robot: is there a table
[INFO] [1775498670.424761, 531.115000]:
```

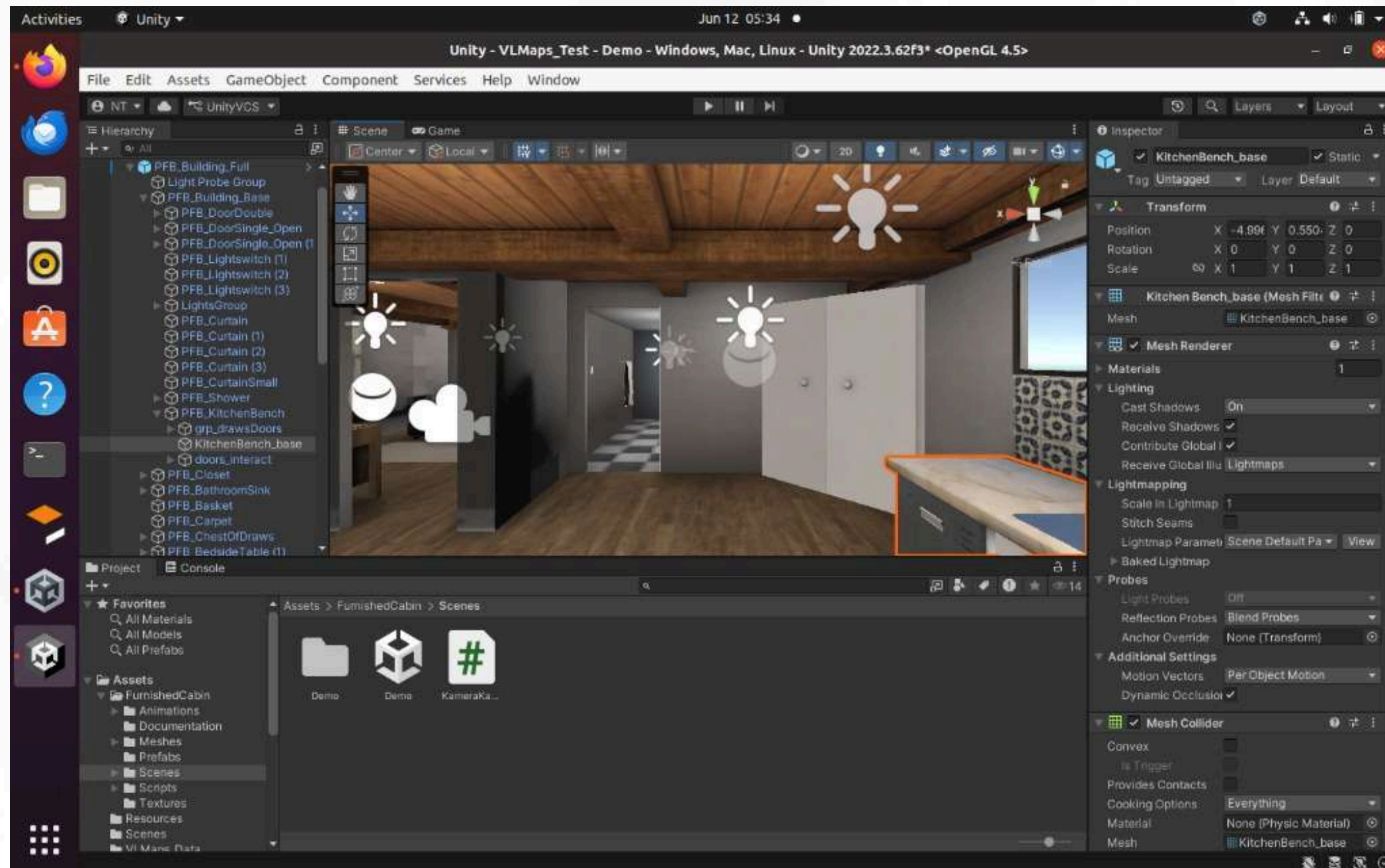
On the right, a camera window titled "TurtleBot3 — Live Camera ..." shows a 3D rendering of a brick wall. The text "Target Lock Scan Active" is overlaid in green at the top left of the camera view.

BLIP provides only general descriptions (“a brick wall”). It can respond with ‘yes’ or ‘no’. It is inadequate for navigation and has a low accuracy rate.

Actual screenshot from our SDP1 mid-term: BLIP captioning a Gazebo scene

SIMULATION ENVIRONMENT: UNITY 3D

DROPPED



Why we dropped it: much higher quality than Gazebo, but too slow on our laptops.

HABITAT-SIM + VLMAPS

CURRENT



- **Photorealistic scenes**
Matterport3D real-room scans
- **RGB-D + camera pose**
Habitat-Sim provides them natively
- **Lab Ubuntu workstation**
GPU sufficient for LSeg inference
- **VLMaps re-implementation**
Builds the actual semantic map
- **Reproducible setup**
Same toolchain as the paper

VLMAPS GENERAL ARCHITECTURE

PHASE 1: PERCEPTION & MAPPING (BUILT ONCE)

This phase creates a "library" of the environment.

(A) VMap Construction: The robot generates a semantic top-down map by fusing RGB, depth, and pose data.

(B) Open-Vocabulary Labeling: The robot is not limited to predefined categories; thanks to the CLIP model, it can identify unlimited natural language terms (e.g., "sofa", "table", "stair") on the map.

(C) Embodiment-Specific Obstacle Maps: Specialized obstacle maps are generated from a single unified map, tailored to the specific physical constraints of different robot types (e.g., Drone vs. Mobile Base).

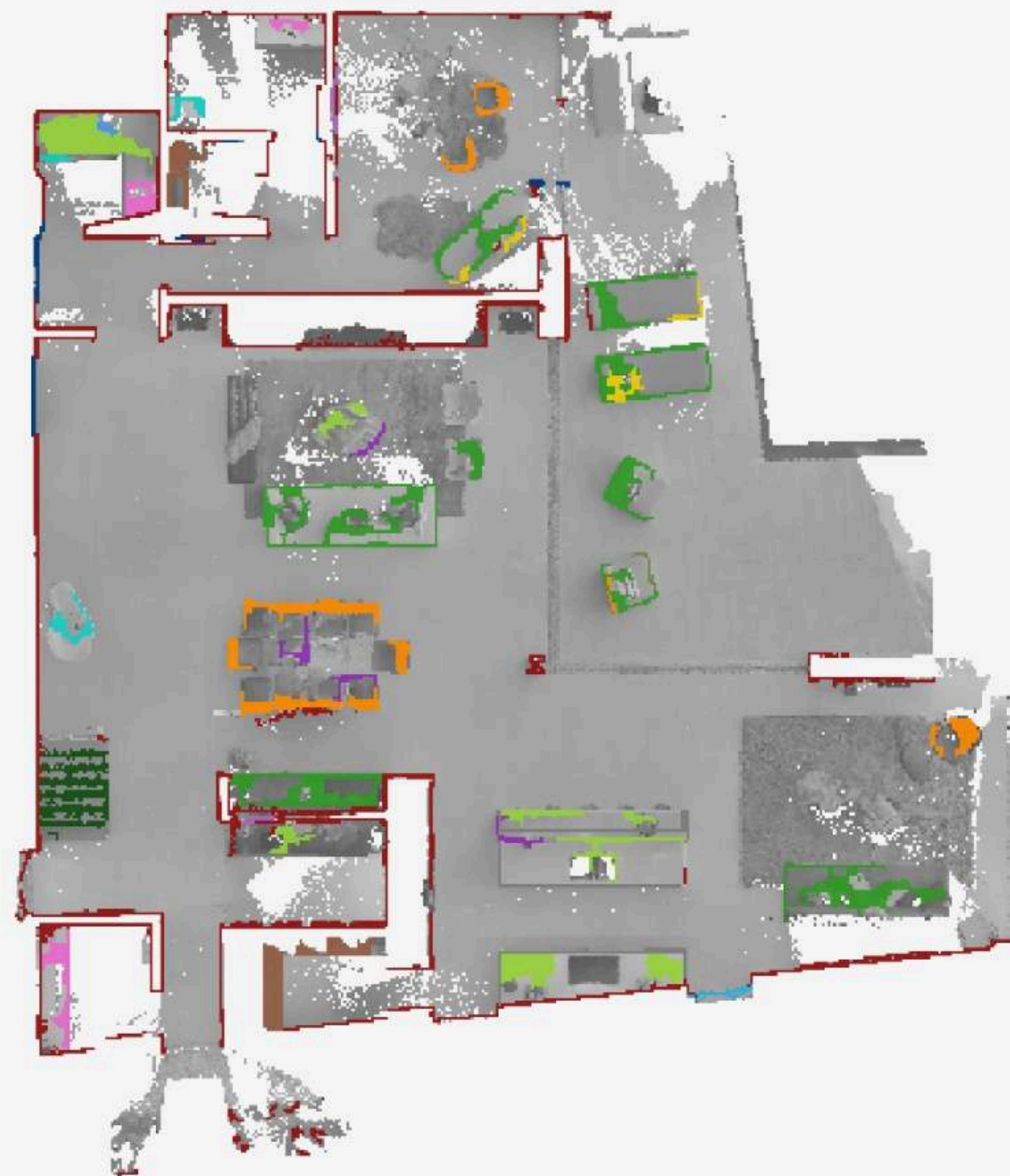
Sequence 1 – RGB Top-Down Reconstruction

Scene: 5LpN3gDmAk7_1



VLMaps Semantic Map

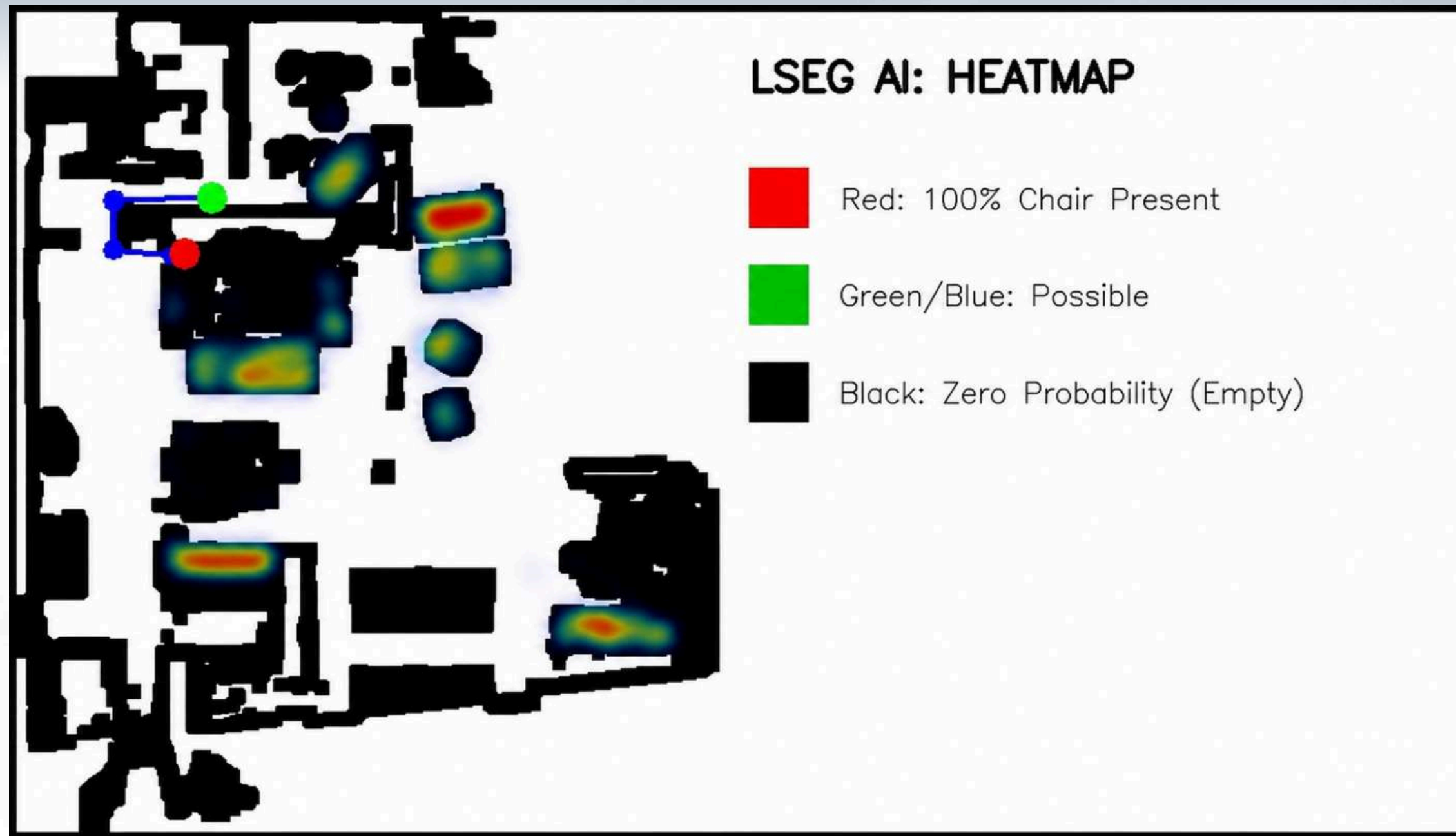
Scene: 5LpN3gDmAk7_1



Detected classes

- wall
- door
- window
- chair
- table
- sofa
- bed
- counter
- cabinet
- shelving
- sink
- toilet
- stairs
- mirror
- towel
- seating

HEAT MAP

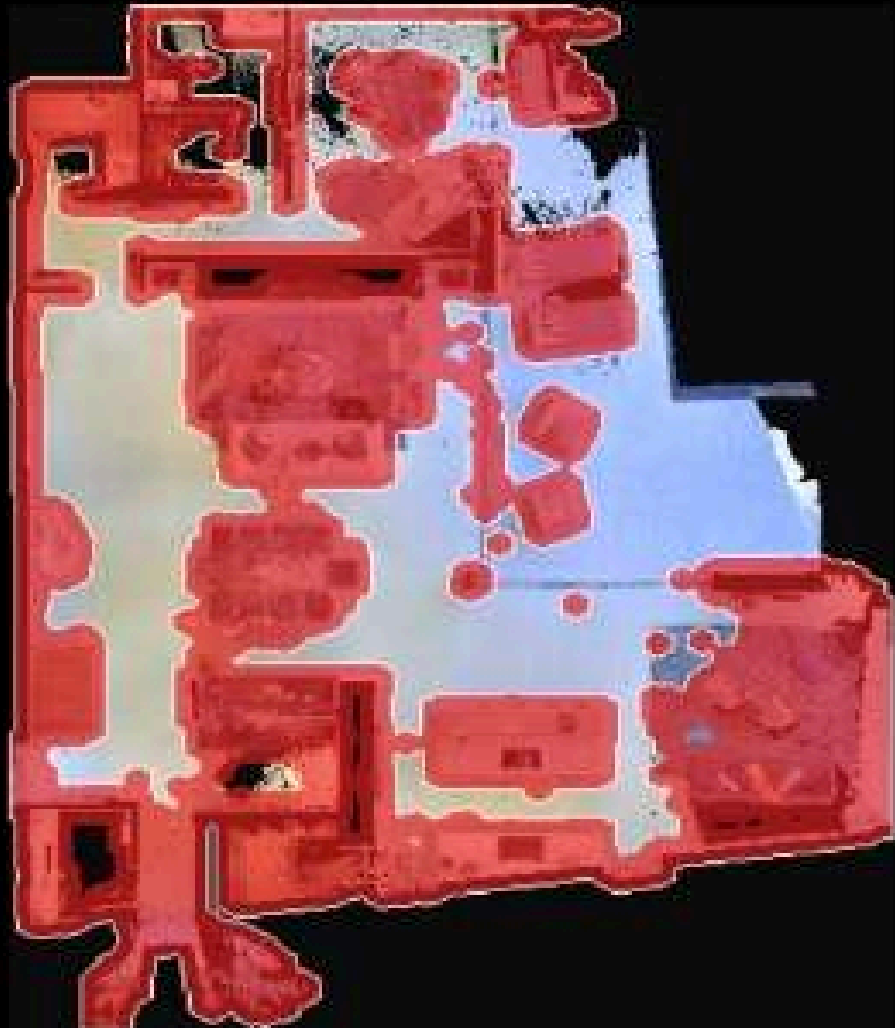


- Similarity over Strict Assignment: Visualizes similarity scores across the map instead of forcing a single class .
- Color Coding: High similarity is shown in warm colors (red); low similarity in cold colors (blue).
- Advantage: Easier interpretation of objects with ambiguous or overlapping boundaries.

CROSS-EMBODIMENT OBSTACLE MAP GENERATION

LoCoBot Obstacle Map

0-1.5 m | exact paper category lists
Obstacle cells: 76785



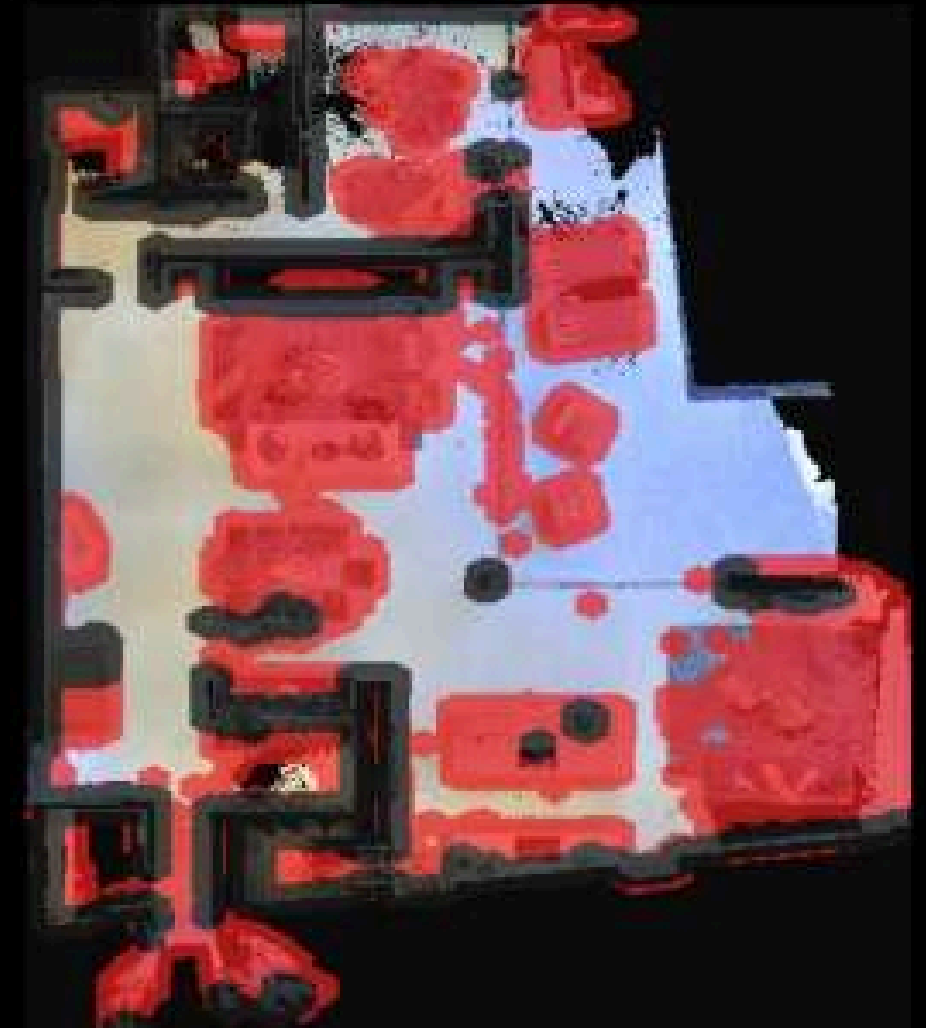
Drone Obstacle Map

1.2-2.2 m band | exact paper category lists
Obstacle cells: 32624



Embodiment Difference

Red: LoCoBot only
Blue: Drone only | Dark: Common



VLMAPS GENERAL ARCHITECTURE

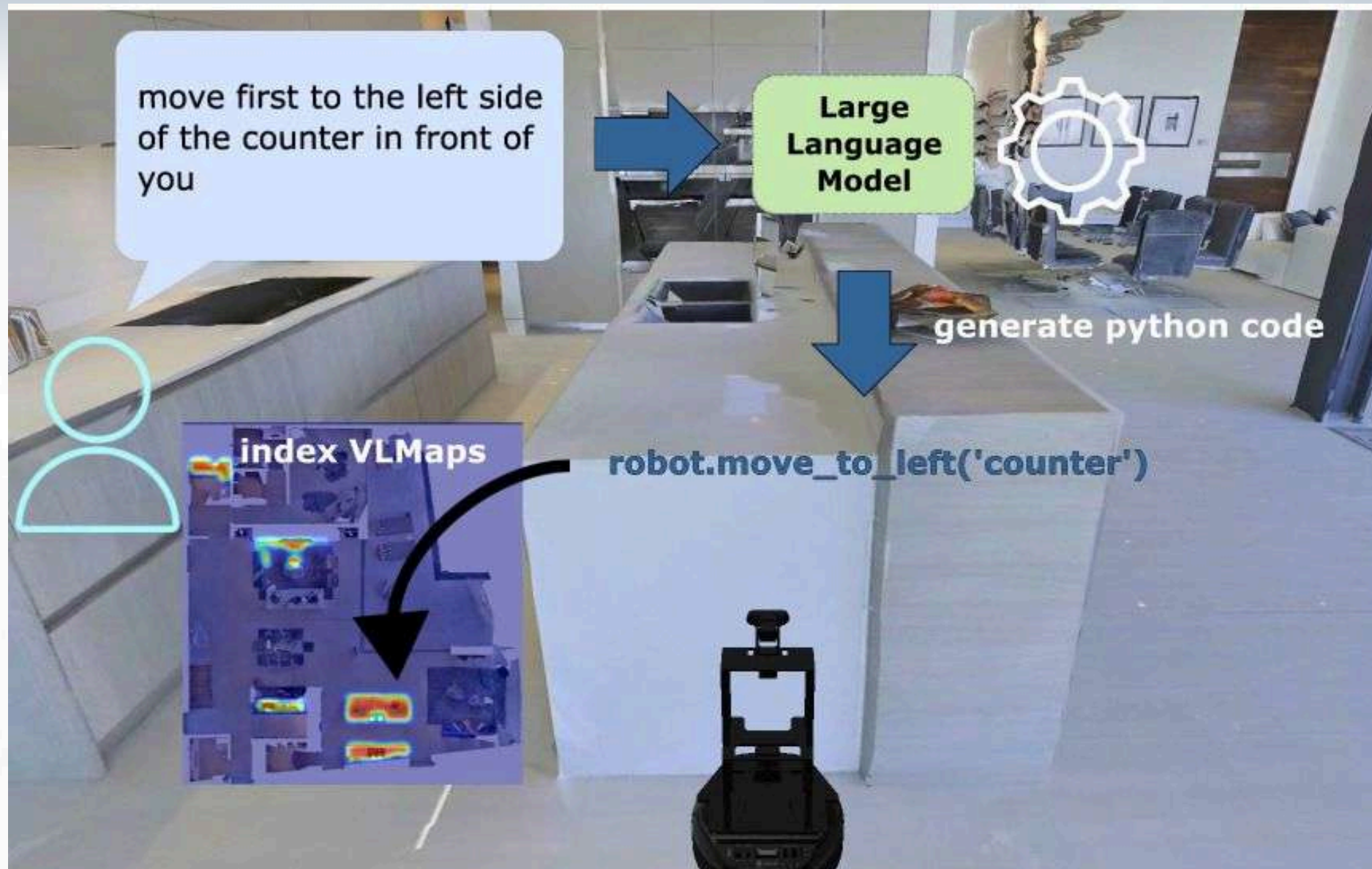
PHASE 2: CONTROL & PLANNING

This phase is where the robot utilizes the map through intelligent reasoning.

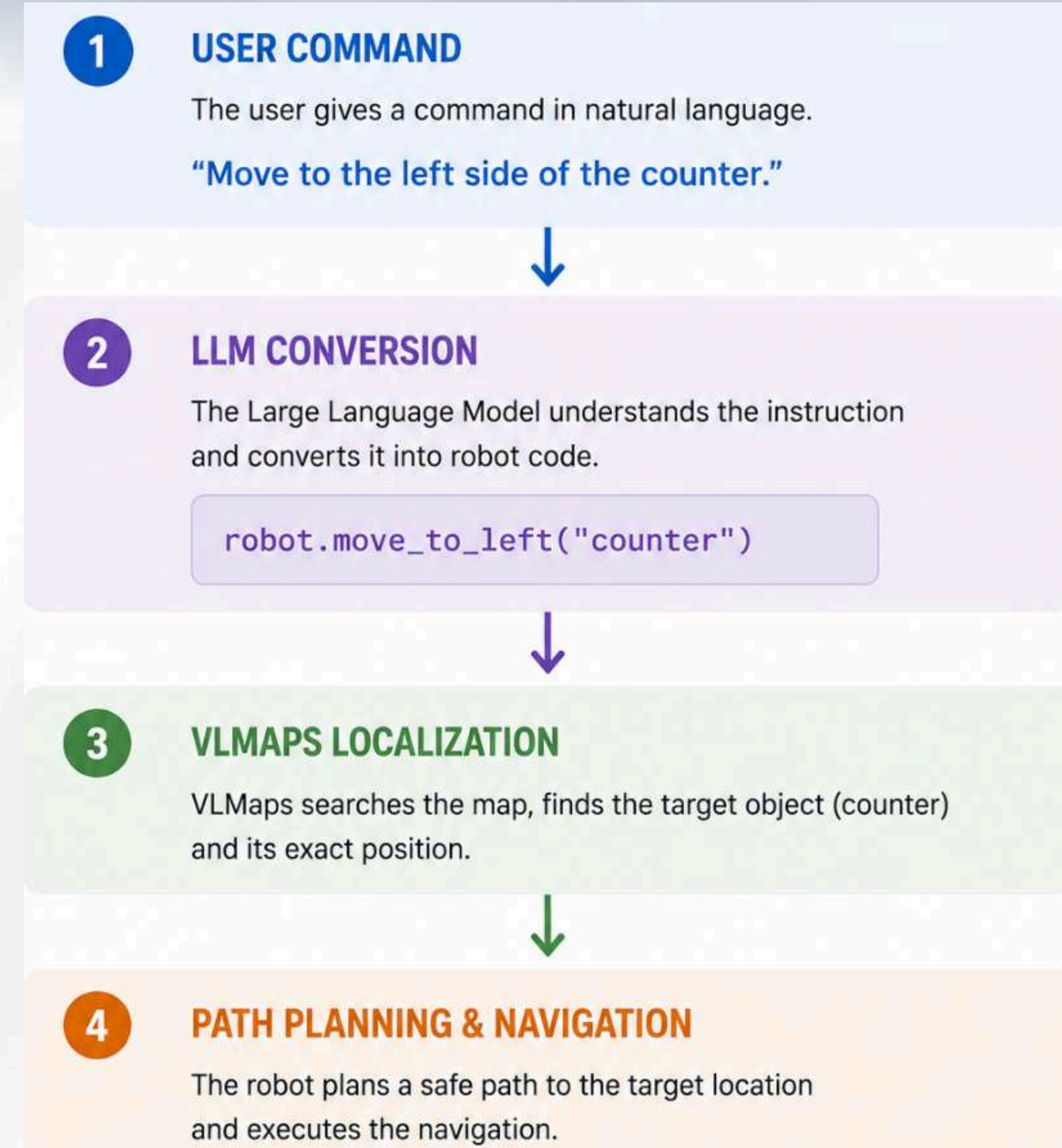
(D)Zero-Shot Goal Navigation: The robot can execute complex, novel tasks on the first try (zero-shot) without requiring any task-specific retraining.

- LLM Command Translation: Natural language commands (e.g., "move to table ") are instantly translated into executable Python code (navigation primitives) via an LLM (Codex).
- Reasoning & Execution: The robot performs high-level reasoning (loops, conditionals) to reach targets on the map autonomously, leveraging the pre-trained knowledge of the language model.

HOW VLMAPS ENABLES ROBOT NAVIGATION?



Huang, C., Mees, O., Zeng, A., & Burgard, W. (2023). Visual Language Maps for Robot Navigation. <https://doi.org/10.48550/arXiv.2210.05714>



GROUND TRUTH MAP

Ground-Truth Semantic Map

Scene: 5LpN3gDmAk7_1 | Boundary-filtered, multi-view semantic projection



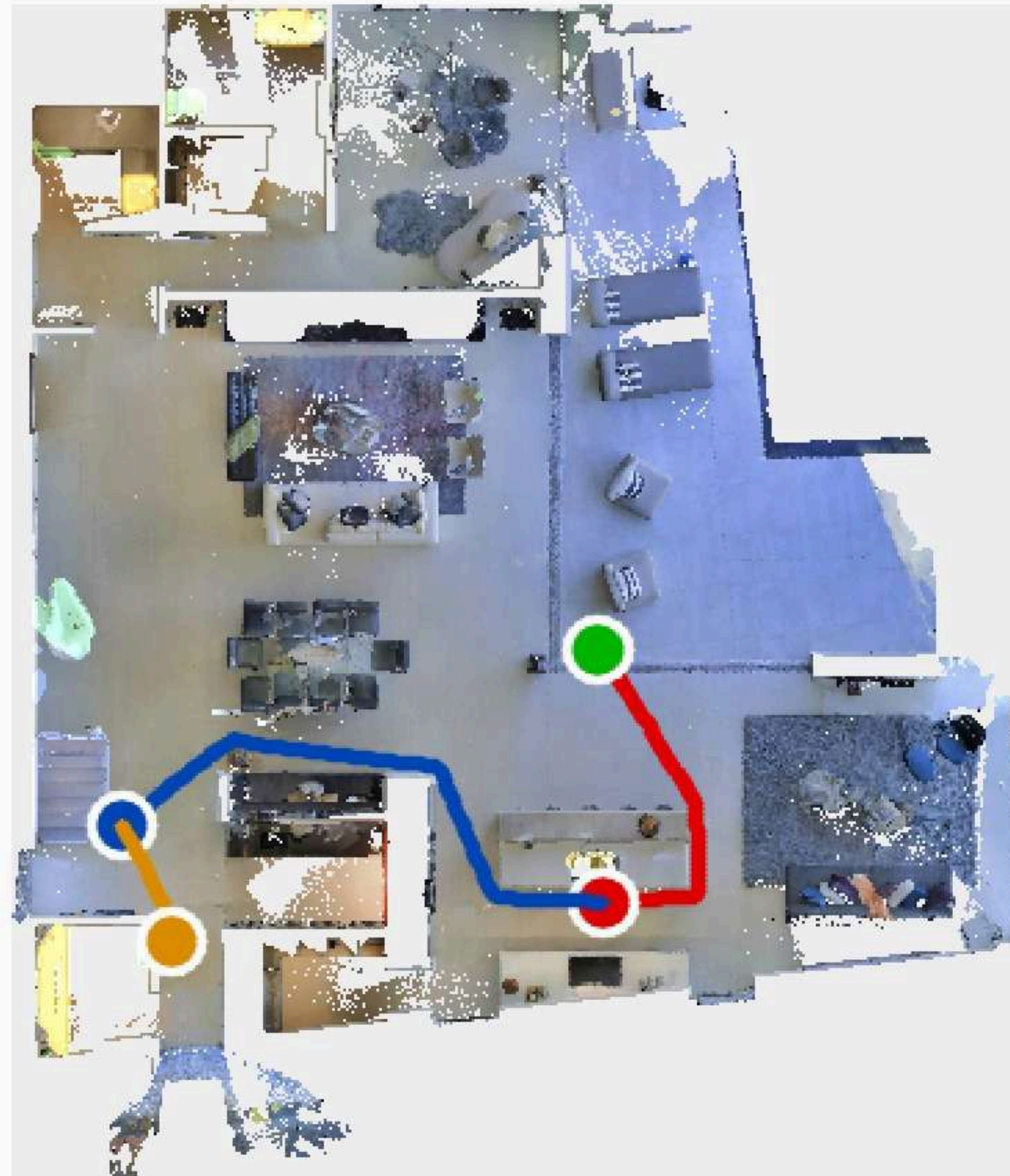
- The Answer Key: The 100% accurate, original map from the simulation (Not generated by VLMaps)
- Zero Noise: No sensor errors; features perfectly sharp boundaries.
- Purpose: The ultimate benchmark to evaluate and compare our system's navigation success.

IMPLEMENTATIONS

SEQUENCE 1- LANGUAGE COMMANDS TO SAFE NAVIGATION PATHS

Custom Language Navigation Result

Trial 32 | GT-scored | hard



Natural-language command

move to the sink, then move to the stairs, then move to the toilet

Generated robot primitive code

```
robot.move_to_object('sink')  
robot.move_to_object('stairs')  
robot.move_to_object('toilet')
```

Navigation metrics

Actions: 289
Path length: 19.79 m
Planning time: 4.35 s
Execution time: 0.56 s
Total measured time: 5.03 s
GT-completed goals: 3/3
In-a-row goals: 3/3
GT final distance: 0.30 m
Predicted-map final distance: 0.40 m

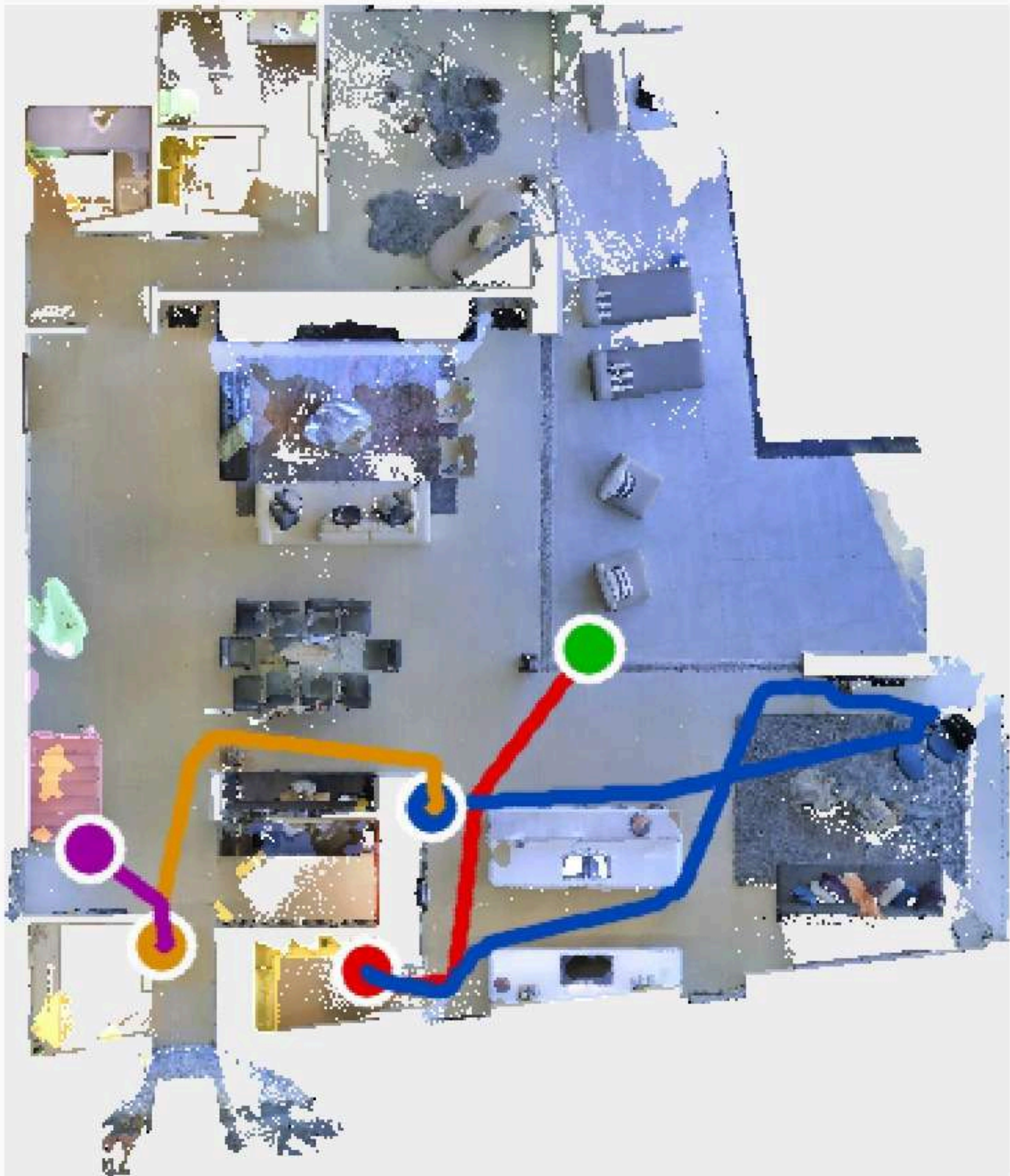
RESULT: SUCCESS

Planning map: VLMaps prediction
Scoring map: validated AABB-derived GT
Language parser: rule-based, not Codex

- Smart Targeting: Identifying and selecting the nearest object instance on the map.
- 100% Success (3/3 goals) in a "Hard" difficulty task.

Custom Language Navigation Result

Trial 34 | GT-scored | very_hard



Natural-language command

move to the shelving, then move to the counter, then move to the toilet, then move to the stairs

Generated robot primitive code

```
robot.move_to_object('shelving')
robot.move_to_object('counter')
robot.move_to_object('toilet')
robot.move_to_object('stairs')
```

Navigation metrics

Actions: 888
Path length: 43.47 m
Planning time: 13.02 s
Execution time: 1.22 s
Total measured time: 14.42 s
GT-completed goals: 4/4
In-a-row goals: 4/4
GT final distance: 0.15 m
Predicted-map final distance: 0.20 m

RESULT: SUCCESS

Planning map: VLMaps prediction
Scoring map: validated AABB-derived GT
Language parser: rule-based, not Codex

50 tasks, distributed by length

1-goal	2-goal	3-goal	4-goal
6 tasks	24 tasks	14 tasks	6 tasks

LoCoBot Obstacle Map

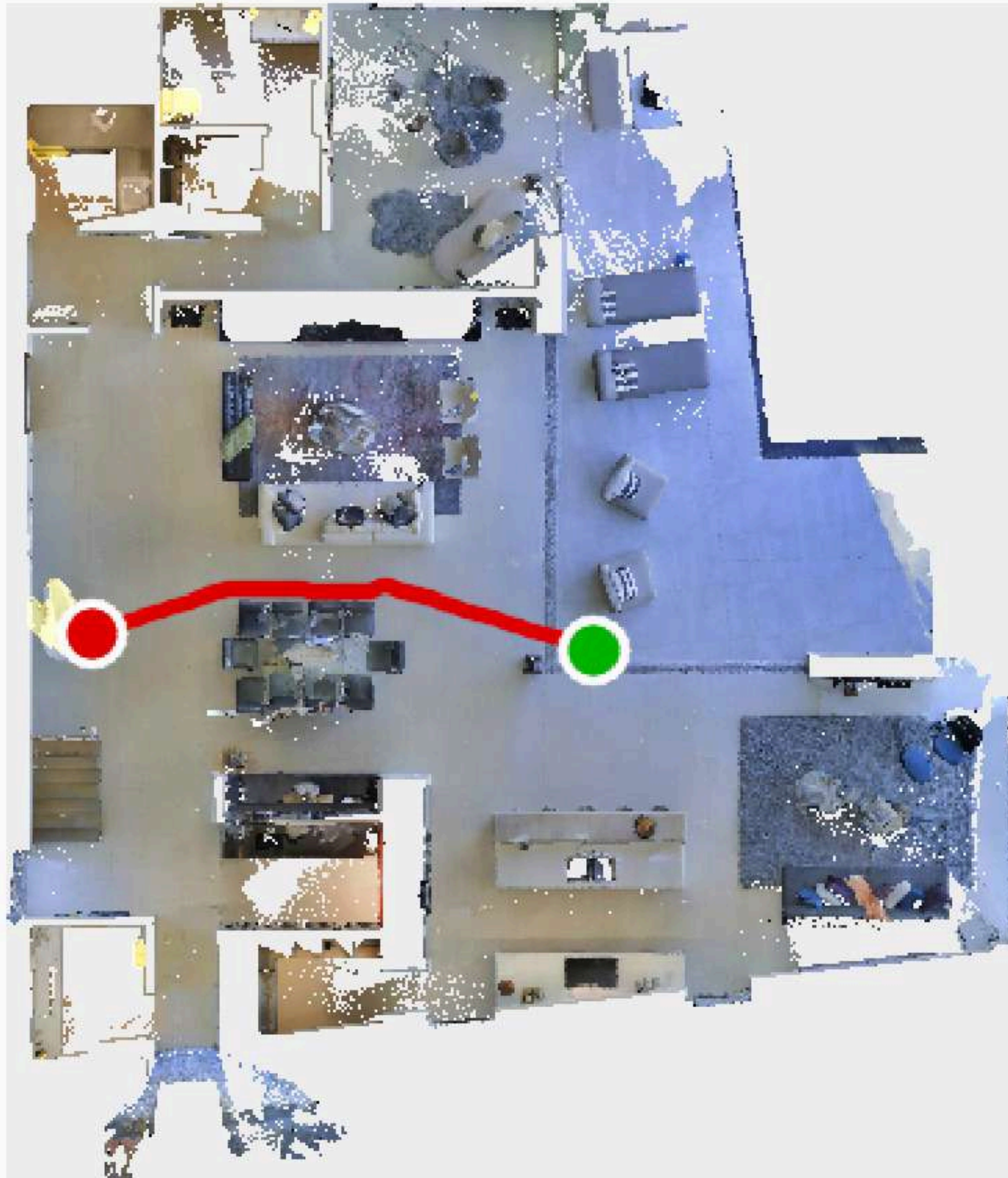
0-1.5 m | exact paper category lists
Obstacle cells: 76785



FAILURE ANALYSIS – SEMANTIC MISLOCALIZATION

Custom Language Navigation Result

Trial 46 | GT-scored | easy



Natural-language command

move to the toilet

Generated robot primitive code

```
robot.move_to_object('toilet')
```

Navigation metrics

Actions: 130

Path length: 9.30 m

Planning time: 1.87 s

Execution time: 0.27 s

Total measured time: 2.18 s

GT-completed goals: 0/1

In-a-row goals: 0/1

GT final distance: 5.51 m

Predicted-map final distance: 0.05 m

RESULT: FAILURE

Planning map: VLMaps prediction

Scoring map: validated AABB-derived GT

Language parser: rule-based, not Codex

PHASE 1 – WRONG TARGET PREDICTION

VLMaps incorrectly labels a chair area as "toilet."



PHASE 2 – CORRECT PATH EXECUTION

The robot follows the planned path and reaches the predicted target.

Predicted-target distance: 0.05 m



PHASE 3 – GROUND-TRUTH EVALUATION

The real toilet is located in a different area.

Ground-truth distance: 5.51 m

RESULT: FAILURE

KEY FINDING

The failure was caused by incorrect semantic localization.

SEQUENCE 3

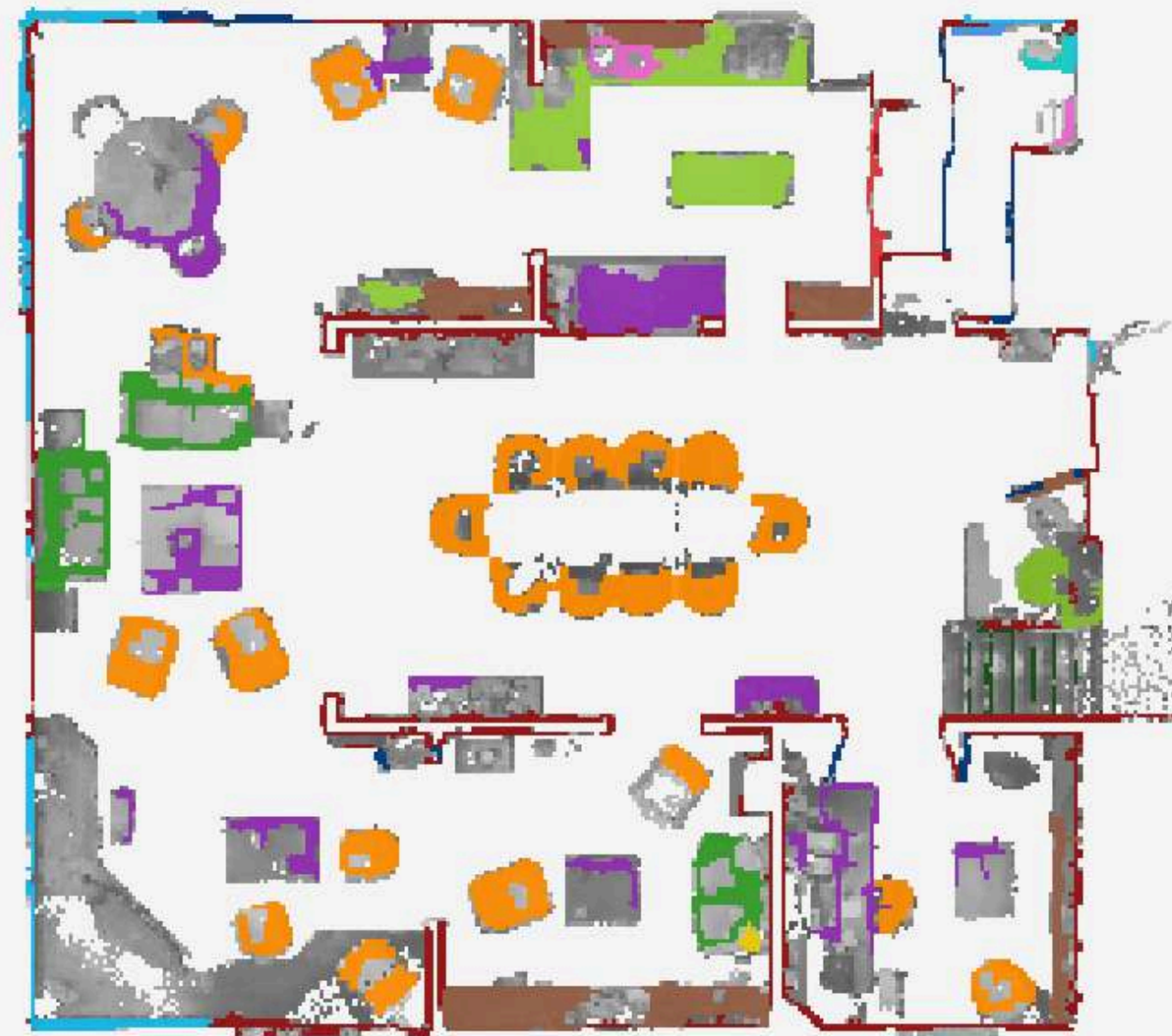
Sequence 3 – RGB Top-Down Reconstruction

Scene: JmbYfDe2QKZ_2



VLMaps Semantic Map

Scene: JmbYfDe2QKZ_2



Detected classes

- wall
- door
- window
- chair
- table
- sofa
- bed
- counter
- cabinet
- shelving
- sink
- toilet
- stairs
- plant
- curtain
- towel
- appliances

GROUND TRUTH FOR SEQUENCE 3

Ground-Truth Semantic Map – RGB Overlay

Scene: JmbYfDe2QKZ_2 | Boundary-filtered, multi-view semantic projection



Semantic classes

- wall
- chair
- door
- table
- picture
- cabinet
- window
- sofa
- plant
- sink
- stairs
- toilet
- towel
- mirror
- tv monitor
- column
- counter
- railing
- shelving
- furniture
- appliances

SEQUENCE 3 – OPEN-VOCABULARY MAPPING TO NAVIGATION

Advanced Language Navigation Result

Trial 04 | Sequence 3 | medium



Natural-language command

move 1.5 meters to the right of the shelving, then move to the table in the lower left

Generated robot code

```
robot.move_relative_to_object('shelving', 'right', 1.5)  
robot.move_to_object_in_region('table', 'lower_left')
```

Overall metrics

Status: completed
Completed goals: 2/2
In-a-row goals: 2/2
Actions: 352
Path length: 17.87 m
Planning time: 4.43 s
Execution time: 0.42 s
Total time: 4.89 s
Logical-start actions: 0

Goal-by-goal evaluation

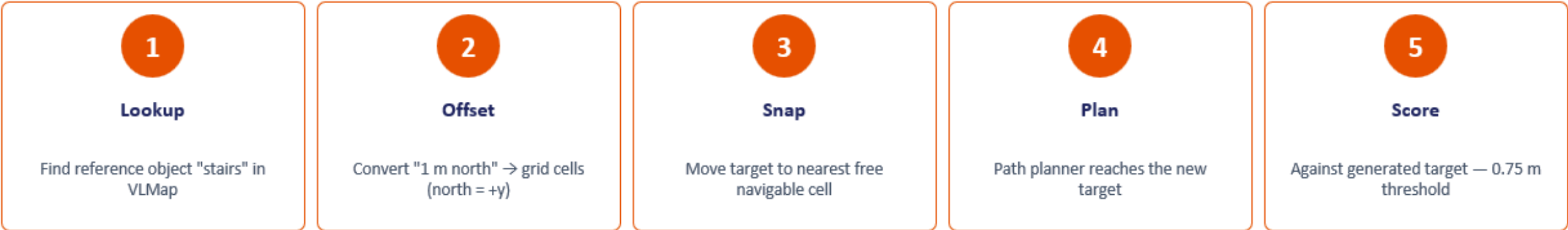
1. move_relative_to_object
SUCCESS | distance=0.32 m | threshold=0.75 m
2. move_to_object_in_region [table]
SUCCESS | distance=0.10 m | threshold=1.00 m

FINAL RESULT: SUCCESS

- Sequence 3 extends object navigation with spatial information.
- The robot must understand directions such as north, south, east, west, left, and right, as well as distances given in meters.
- **Main focus:** Precise position, direction, distance, and orientation.

SEQUENCE 3 – OPEN-VOCABULARY MAPPING TO NAVIGATION

HOW THE METRIC TARGET IS GENERATED — five sub-steps



Sequence-1 Baseline + Metric Relative Navigation

Trial 37 | Sequence 3 | medium



Natural-language command

move 1 meter to the north of the stairs, then move to the chair

Generated robot code

```
robot.move_relative_to_object('stairs', 'north', 1.0)
robot.move_to_object('chair')
```

Navigation metrics

Actions: 157
Path length: 10.30 m
Planning time: 2.46 s
Execution time: 0.29 s
Completed goals: 2/2
In-a-row goals: 2/2

Goal-by-goal evaluation

- 1. stairs
SUCCESS | distance=0.30 m | limit=0.75 m
- 2. chair
SUCCESS | distance=0.20 m | limit=1.00 m

RESULT: SUCCESS

FAILURE ANALYSIS – VISUAL-SIMILARITY CONFUSION

Sequence—1 Baseline + Metric Relative Navigation

Trial 45 | Sequence 3 | hard



Natural-language command

move to the stairs, then move to the sofa, then move to the sink

Generated robot code

```
robot.move_to_object('stairs')  
robot.move_to_object('sofa')  
robot.move_to_object('sink')
```

Navigation metrics

Actions: 347
Path length: 19.47 m
Planning time: 6.01 s
Execution time: 0.85 s
Completed goals: 2/3
In-a-row goals: 1/3

Goal-by-goal evaluation

- 1. stairs
SUCCESS | distance=0.38 m | limit=1.00 m
- 2. sofa
FAILURE | distance=6.02 m | limit=1.00 m
- 3. sink
SUCCESS | distance=0.70 m | limit=1.00 m

RESULT: FAILURE

1 PHASE 1 — FIRST GOAL COMPLETED

The robot successfully reaches the stairs.

Distance to stairs: 0.38 m — SUCCESS

2 PHASE 2 — SOFA QUERY

For the word “sofa,” VLMaps performs a separate visual-language similarity search.

The target is not selected only from the colored semantic-map labels.

3 PHASE 3 — WRONG TARGET SELECTION

Because chairs and sofas have similar visual features, a chair region receives a higher similarity score.

The robot navigates correctly, but toward the wrong target.

4 PHASE 4 — GROUND-TRUTH EVALUATION

The robot stops 6.02 m away from the real sofa.

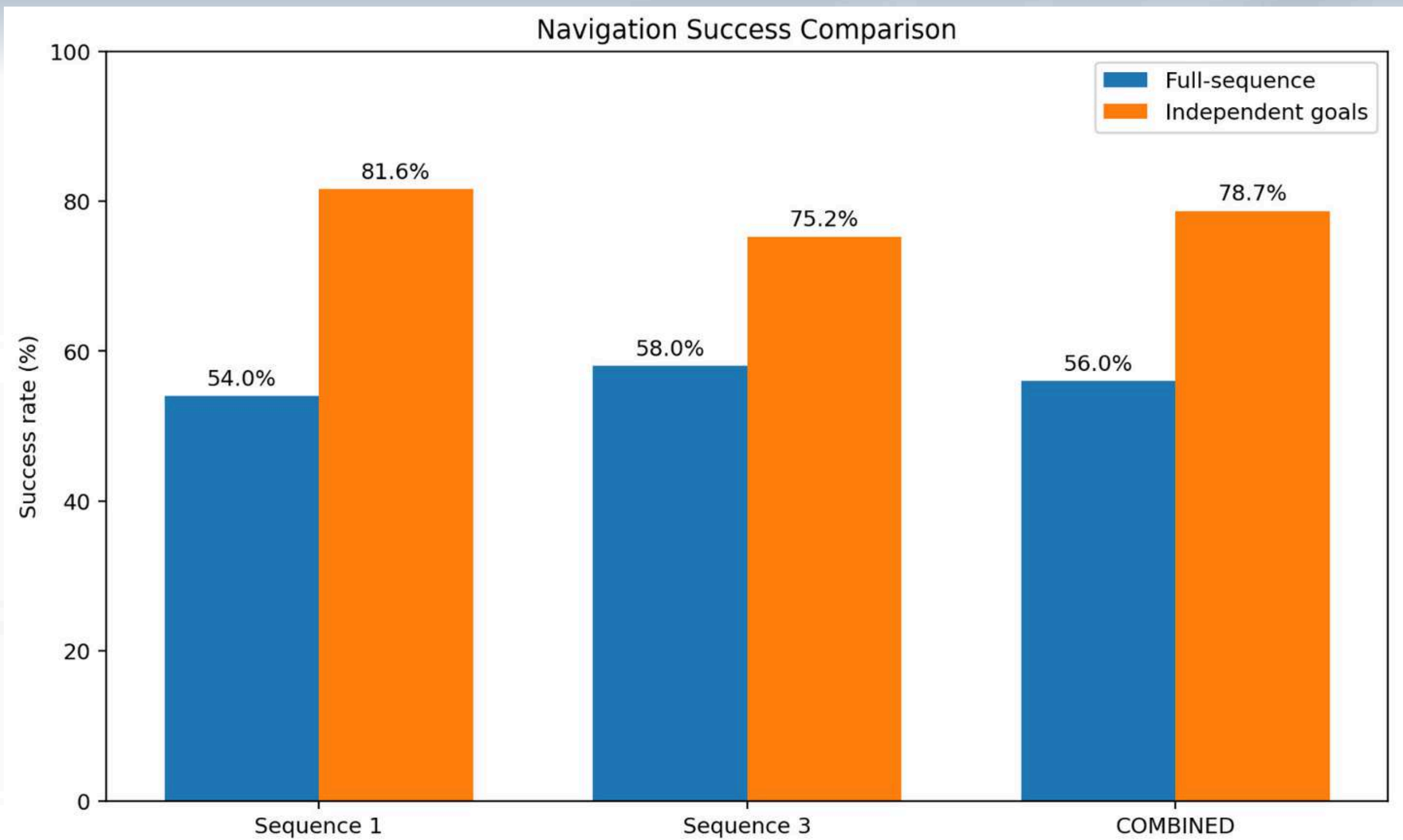
Success limit: 1.00 m

SOFA GOAL: FAILURE

The robot then successfully reaches the sink.

Distance to sink: 0.70 m — SUCCESS

NAVIGATION SUCCESS COMPARISON



Tasks	No. Subgoals in a Row				Independent Subgoals
	1	2	3	4	
LM-Nav [13]	26	4	1	1	26
CoW [12]	42	15	7	3	36
CLIP Map	33	8	2	0	30
VLMs (ours)	59	34	22	15	59
GT Map	91	78	71	67	85

Huang, C., Mees, O., Zeng, A., & Burgard, W. (2023). Visual Language Maps for Robot Navigation. <https://doi.org/10.48550/arXiv.2210.05714>

Figure : The results of 100 tests of varying difficulty

TIMELINE AND WORK PACKAGES



Work package name	TIMELINE (Months)									
	Fall Term					Spring Term				
	9	10	11	12	01	02	03	04	05	06
Literature Review						X	X			
Setting up and configuring the VLM on a local development environment (Laptop)							X	X		
Testing perception and reasoning modules in Gazebo simulation							X	X		
Development of robot control and localization algorithms within ROS simulations							X	X		
Integration of VLM-based semantic data with robot control algorithms								X	X	
Transitioning and optimizing the software stack for the Jetson Nano board	X								X	X
Real-world performance testing and validation on mobile robot platforms		X	X	X						
Analysis of test results, repetition of low-performance tests with new methods, and writing a publishable conference paper.			X	X	X					

NEXT STEPS

NEXT STEP: REAL-WORLD DATA COLLECTION AND VLMap CREATION

A. DATA COLLECTION PIPELINE

- 1 RGB-D Camera Setup**
Use an Intel RealSense D435i to capture RGB and depth images.
- 2 RTAB-Map Pose Estimation**
RTAB-Map estimates the camera pose during data collection.
- 3 ROS Bag Recording**
RGB images, depth images, and camera poses are recorded in a .bag file on WSL2 Ubuntu 22.04.
- 4 Data Extraction**
A script extracts the bag contents into: RGB images, depth images, and a poses.txt file.
- 5 Dataset Packaging**
The extracted folder is compressed as a ZIP file and uploaded to Google Drive.

B. VLMap CREATION PIPELINE

- 1 Google Colab Setup**
The ZIP dataset is opened in Google Colab because VLMap creation requires high memory and GPU resources.
- 2 CLIP Feature Extraction**
CLIP is applied to RGB images to extract visual-language features.
- 3 LSeg Segmentation**
LSeg is used to obtain pixel-level semantic segmentation.
- 4 Point Cloud Generation**
RGB-D images and camera poses are combined to build a 3D point cloud map.
- 5 Pose Validation**
Different pose settings are tested to find the correct camera-map alignment.
- 6 Top-Down Map Generation**
A 2D bird's-eye-view map is created from the 3D reconstruction.
- 7 VLMap Generation**
CLIP features are fused into the map cells to create the final VLMap.

Goal: Transfer the VLMaps pipeline from simulation to real RGB-D data collected with a RealSense camera.

NEXT STEPS



The experimentally derived semantic map of the robotics laboratory

THANK YOU

